- By default, you can't bind the body of `GET` requests, as that goes against the expectations for `GET` requests. Doing so will cause an exception at runtime.
- Arrays of simple types bind by default to query string values for `GET` requests and to the request body for `POST` requests. This difference can cause confusion, so always consider whether an array is the best option.
- All the parameters of a handler must bind correctly. If a parameter tries to bind to a missing value, you'll get a `BadHttpRequestException` and a `400 Bad Request` response.
- You can use well-known types such as `HttpContext` and any services from the dependency injection container in your endpoint handlers. Minimal APIs check whether each complex type in your handler is registered as a service in the DI container; if not, they treat it as a complex type to bind to the request body instead.
- You can read files sent in the request by using the `IFormFile` and `IFormFileCollection` interfaces in your endpoint handlers. Take care accepting file uploads with these interfaces, as they can open your application to attacks from users.
- You can completely customize how a type binds by using custom binding. Create a static function with the signature `public static ValueTask<T?> BindAsync(HttpContext context)`, and return the bound property. This approach can be useful for handling complex scenarios, such as arbitrary JSON uploads.